

Design and Implementation of a Smart System for Bank Marketing Prediction

An Industrial Machine Learning Approach

Mohammad Amin Haji Alirezai¹, Hamid Reza Mortazavi¹, and
Mohammad Sajjad Hamidi Fard¹

¹Supervisor: Dr. Boshra Pishgoo

Autumn 2025 – Winter 2026

Project Overview

This project is defined under the Industrial Machine Learning path. The primary objective is to develop a predictive model to identify customer interest in bank term deposits based on demographic and interaction data.

Contents

1	Selected Path	4
2	Problem Definition	4
2.1	Success Criteria	4
2.2	Business Context	4
2.3	Machine Learning Formulation	5
3	Appropriate Dataset Selection	5
3.1	Data Profile	5
3.2	Inherent Data Challenges	5
3.3	Feature Dictionary	5
3.3.1	Client Profile (Demographics)	6
3.3.2	Last Contact Attributes	6
3.3.3	Campaign History and Social-Economic Context	6
3.3.4	Target Variable	6
4	Exploratory Data Analysis (EDA)	7
4.1	Class Imbalance Analysis	7
4.2	Numerical Variables and Outliers Analysis	7
4.3	Correlation and Leakage Warning	7

5	Data Preprocessing	7
5.1	Data Cleaning	7
5.2	Data Transformation	7
5.3	Encoding	8
6	Baseline Model Design	8
6.1	Selection Rationale	8
6.2	Training Configuration	8
7	Experiment Execution and Evaluation	8
7.1	Evaluation Metrics	8
7.2	Outputs and Artifacts	9
8	Implementation of SMOKE TEST	10
8.1	Implemented Test Architecture	10
8.1.1	a) Infrastructure Validation	10
8.1.2	b) Training Stability Checks	10
8.1.3	c) Inference Capability	11
8.2	Continuous Integration (CI/CD)	11
9	Random Forest (Advanced Results & Comparison)	11
9.1	Configuration and Model Robustness	11
9.2	Performance Benchmarking Table	12
9.3	Feature Importance Analysis	12
9.4	Strategic Insight	12
10	Transition to Advanced Models (Advanced Modeling with XGBoost)	12
10.1	Experiment 1: XGBoost with SMOTE	13
10.2	Bayesian Optimization (Optuna) on SMOTE Data	13
11	Weighted Modeling Paradigm Shift (Weighted XGBoost)	14
11.1	Mathematical Rationale	15
11.2	The Breakthrough	15
12	Final Optimization & Fine-Tuning	15
12.1	Combining Weighting and Optuna	15
12.2	Threshold Tuning	16
13	Design and Implementation of User Interface and Interactive System	16
13.1	Introduction	16
13.2	System Architecture and Technologies	16
13.3	User Interface Structure	17
13.3.1	Input Information Panel	17
13.4	Prediction Dashboard and Model Comparison	17
13.5	Intelligent Recommender System	17
13.6	Model Explainability with SHAP	18
13.6.1	Tab 1: Local Explanation	19
13.6.2	Tabs 2 & 3: Feature Contributions and Global Importance	19
13.7	Conclusion	20

14 Final Performance Comparison	21
14.1 Summary	21
15 Comprehensive Evaluation & Explainability	22
15.1 Comparative ROC Analysis	22
15.2 Confusion Matrix Breakdown	22
15.3 Scientific Explainability with SHAP	23
15.3.1 a) Key Drivers Analysis (Summary Plot)	23
15.3.2 b) Individual Customer Interpretation (Force Plot)	23
15.4 Chapter Conclusion	23
16 Resources and Project Links	25
17 Technology Stack	25

1 Selected Path

This project is defined within the framework of the Classic Machine Learning path. The primary reasons for choosing this path include:

- **Strategic Importance:** The transition of banks from mass marketing to data-driven, personalized marketing is a key application of classical models in the industry.
- **Technical Challenges:** This project presents a combination of real-world data science challenges, such as imbalanced data, mixed features, and the requirement for interpretability. Mastering these foundational algorithms is highly valuable for industrial applications.

2 Problem Definition

The goal of this project is to detect the inclination of customers toward a "Deposit Subscription" based on demographic information and contact history.

2.1 Success Criteria

To consider the model successful, it must achieve the following performance metrics:

- **Accuracy:** $\geq 85\%$.
- **F1-score:** ≥ 0.45 .
- **ROC-AUC:** To measure the model's ability to distinguish between classes regardless of the decision threshold.
- **Recall:** Critical for ensuring the bank does not miss potential customers.
- **Precision:** Important for reducing the operational costs associated with contacting uninterested individuals.

2.2 Business Context

In the modern banking ecosystem, term deposits are a vital source of liquidity. Traditional strategies like mass telemarketing face two major hurdles:

- **High Operational Costs:** Calling customers with no interest wastes time and human resources.
- **Negative User Experience:** Frequent, non-targeted calls can lead to customer dissatisfaction and churn.

2.3 Machine Learning Formulation

The problem is formulated as a **Supervised Binary Classification** task. The target variable $y \in \{0, 1\}$ represents:

- **1 (Success):** The customer opens a deposit account.
- **0 (Failure):** The customer rejects the offer.

The feature space includes client demographics, financial records, and previous interaction history.

3 Appropriate Dataset Selection

For this research, the standard "Bank Marketing" dataset from the University of California, Irvine (UCI) Machine Learning Repository was utilized.

3.1 Data Profile

- **Dataset Title:** Bank Marketing Data Set.
- **Source:** Moro et al. (2014) - related to a Portuguese banking institution.
- **Volume:** 45,211 instances in the full version (bank-full.csv).
- **Dimensions:** 16 input features + 1 target variable.
- **Data Types:** Multivariate, including a mix of numerical and categorical variables.

3.2 Inherent Data Challenges

Identifying challenges prior to modeling is critical for a researcher:

- **Severe Class Imbalance:** Distribution is approximately 88% negative and 12% positive. This can lead the model to overfit on the majority class. Strategies like SMOTE or Cost-Sensitive Learning are necessary.
- **Data Leakage Risk (Duration):** The "duration" feature is highly correlated with the target as longer calls usually imply success. However, this value is unknown before a call is made, making its use in real-time prediction a form of data leakage.
- **Missing Values:** While no explicit null values exist, missing information is encoded as "unknown" in categorical variables like job and education, requiring intelligent imputation strategies.

3.3 Feature Dictionary

The variables in this dataset are categorized as follows:

Feature Name	Data Type	Description
age	Numeric	Client's age.
job	Categorical	Type of job (admin, technician, student, etc.).
marital	Categorical	Marital status (married, single, divorced).
education	Categorical	Education level (primary, secondary, tertiary).
default	Binary	Does the client have credit in default?.
balance	Numeric	Average annual balance (in Euros).
housing	Binary	Does the client have a housing loan?.
loan	Binary	Does the client have a personal loan?.

Feature Name	Data Type	Description
contact	Categorical	Contact communication type (cellular, telephone).
day	Numeric	Last contact day of the month.
month	Categorical	Last contact month of the year.
duration	Numeric	Last contact duration in seconds (leakage potential).

Feature Name	Data Type	Description
campaign	Numeric	Number of contacts performed during this campaign.
pdays	Numeric	Days since last contact from a previous campaign.
previous	Numeric	Number of contacts performed before this campaign.
poutcome	Categorical	Outcome of the previous marketing campaign.

3.3.1 Client Profile (Demographics)

3.3.2 Last Contact Attributes

3.3.3 Campaign History and Social-Economic Context

3.3.4 Target Variable

Feature Name	Data Type	Description
y	Binary	Did the client subscribe to a term deposit? (yes/no)

4 Exploratory Data Analysis (EDA)

A comprehensive EDA was conducted to understand the data structure, identify hidden patterns, and prepare for modeling.

4.1 Class Imbalance Analysis

Success (Yes) and Failure (No) distributions were visualized using bar and pie charts. The analysis confirmed severe imbalance, justifying the need for techniques like SMOTE or Class Weighting in the modeling phase.

4.2 Numerical Variables and Outliers Analysis

Distributions were evaluated for skewness and outliers:

- **Age:** Histograms showed the age distribution of the banking clients.
- **Balance:** Using Boxplots and the Interquartile Range (IQR) method, extreme outliers in account balances were identified for potential removal or normalization.

4.3 Correlation and Leakage Warning

A Pearson correlation matrix and heatmap were generated to identify linear relationships and prevent multicollinearity. Crucially, the "duration" variable showed a very high correlation with success. Since this value is only known after the call concludes, it constitutes "Data Leakage". Consequently, according to industrial standards, the decision was made to remove this variable from the training process.

5 Data Preprocessing

Based on the "Garbage In, Garbage Out" principle and the analyses performed in the previous section, the following strategies were adopted to prepare the data.

5.1 Data Cleaning

- **Missing Values Management:** For variables like `job` and `education`, 'unknown' values are preserved as a distinct class, as the "unknown" status itself may contain information.
- **Outlier Management:** The `balance` variable exhibits severe outliers. The Interquartile Range (IQR) method was used to identify and remove records significantly distant from the mean.

5.2 Data Transformation

Standardization was applied to numerical features (`age`, `balance`, `day`, `campaign`) using the formula:

$$z = \frac{X - \mu}{\sigma} \tag{1}$$

Reasoning: Since the scale of `balance` differs significantly from `age`, failure to standardize would cause algorithms relying on distance calculations to perform poorly.

5.3 Encoding

- **One-Hot Encoding:** Applied to nominal variables without inherent order: `job`, `marital`, `contact`, `poutcome`.
- **Label/Ordinal Encoding:** Applied to `education` (primary ; secondary ; tertiary) and `month` (chronological order).
- **Reasoning:** Preserving the intrinsic order helps the model better understand trend-based patterns.

6 Baseline Model Design

To establish a benchmark for measuring performance improvements in complex models and evaluating preprocessing quality, **Logistic Regression** was selected for this phase.

6.1 Selection Rationale

- **Compatibility with Preprocessing:** Given the use of Standardization for numerical features, this linear model (which operates based on distance and gradients) shows the best compatibility with the data distribution.
- **Interpretability:** The transparency of coefficients allows for an initial analysis of feature impact (e.g., the direct or inverse relationship of `age` and `balance` with the target).
- **Linearity Assessment:** This model serves as a metric to determine if the data is linearly separable or if non-linear models like XGBoost are strictly necessary.

6.2 Training Configuration

In the implementation of the `train_baseline.py` script, the following settings were applied:

- **Imbalance Handling:** The parameter `class_weight='balanced'` was used. This allows the model to automatically assign higher weight to the minority class (customers who opened a deposit), preventing severe bias toward the negative class.
- **Convergence Guarantee:** Given the data volume, `max_iter=1000` was set to ensure the `lbfgs` optimization algorithm has sufficient opportunity to fully converge.

7 Experiment Execution and Evaluation

7.1 Evaluation Metrics

Aligned with the goals defined in Section 2.3, the baseline model was assessed on the **Validation Set** using the following metrics:

- **Recall (Class 1):** The most critical metric to ensure the bank does not miss opportunities to acquire interested customers.
- **Precision:** To control costs associated with erroneous calls.
- **ROC-AUC:** To measure the model's discriminative power independent of the decision threshold.
- **Accuracy:** Monitored for general overview, though less significant due to class imbalance.

7.2 Outputs and Artifacts

Upon execution of the training code, the following outputs were automatically generated and saved:

- **Serialized Model:** The file `baseline_logreg.pkl` was saved in the models directory for reuse or deployment.
- **Visual Analysis:**
 - **Confusion Matrix:** To observe the exact count of False Positives (FP) and False Negatives (FN).
 - **ROC Curve:** To visually display model performance and calculate the Area Under the Curve.

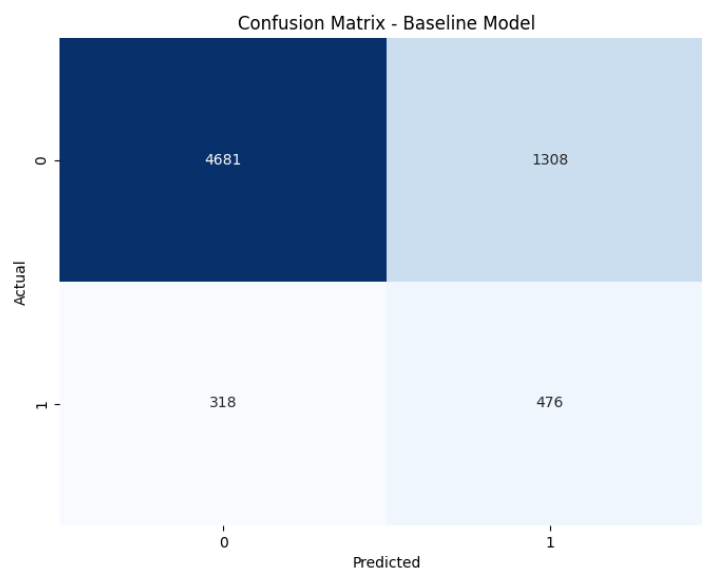


Figure 1: Confusion Matrix for Baseline Logistic Regression Model

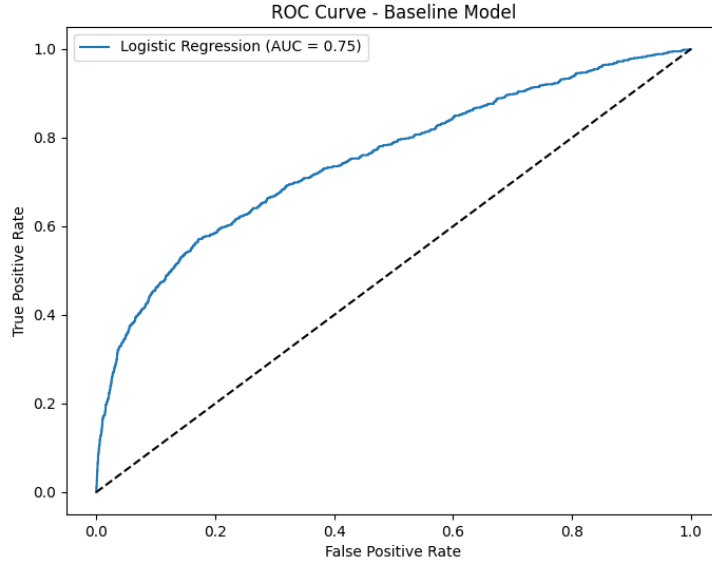


Figure 2: ROC Curve for Baseline Model (AUC = 0.75)

8 Implementation of SMOKE TEST

In operational scales, coding machine learning models is only half the journey. The other half is guaranteeing that system integrity is maintained as model complexity increases from Logistic Regression to XGBoost. To this end, a comprehensive **Smoke Testing** framework was developed to automatically verify not just code correctness, but also the modeling logic across all phases.

8.1 Implemented Test Architecture

The script `tests/test_smoke.py` was redesigned to act as a supervisor for all project modules. It creates an isolated environment with Mock Data to check seven critical scenarios in every run:

8.1.1 a) Infrastructure Validation

- **Configuration Test (`test_01_configuration`):** Ensures the critical `config.yaml` file is accessible and key parameters like data paths are correctly defined.
- **Preprocessing Health and Data Leakage Check (`test_02_preprocessing`):** Calls the `load_and_preprocess_data` function. Crucially, it includes a **Security Control** to automatically verify that the `duration` column (the leakage factor) has been removed from the final output. This serves as a vital Guardrail against human error in the operational environment.

8.1.2 b) Training Stability Checks

Unlike standard unit tests, this section ensures that training codes for various models execute without crashing. These tests run on small data subsets for speed:

- **Baseline Model (test_03):** Checks the health of the Logistic Regression training code.
- **Tree Models (test_04):** Verifies the correct execution of the Random Forest pipeline, including the SMOTE technique and Cross-Validation (CV).
- **Gradient Models (test_05):** Tests the XGBoost pipeline execution and handles potential errors in the Boosting process.
- **Champion Model (test_06):** Checks the final and most complex project model (Weighted XGBoost) to ensure the final product version is always reproducible.

8.1.3 c) Inference Capability

- **Operation Simulation (test_07):** In this final test, the "Champion" model saved in the previous step is reloaded to perform predictions on a new data sample. This guarantees that the output .pkl files are healthy and the final API will not encounter errors at runtime.

8.2 Continuous Integration (CI/CD)

The file `smoke-test.yml` acts as a "Gatekeeper" in the GitHub repository. With every code change:

1. A Python 3.10 environment is created on cloud servers.
2. All dependencies are installed.
3. The aforementioned test suite is executed.

Technical Result: This infrastructure assures that even if complex changes are applied to the XGBoost code, they will not enter the final stage unless all 7 tests are passed.

9 Random Forest (Advanced Results & Comparison)

This section analyzes the results of the industrial Random Forest model implementation, compares its performance with the Baseline model, and examines the impact of the SMOTE technique in resolving the data imbalance challenge.

9.1 Configuration and Model Robustness

To ensure scientific accuracy and generalizability, the following configuration was applied:

- **Intelligent Pipeline (ImbPipeline):** Used the `imblearn` library pipeline to ensure SMOTE is applied *only* to the training data of each fold, preventing data leakage.
- **Stratified CV (5-Fold):** Implementation of Cross-Validation resulted in a mean F1-score of 0.8698, indicating model stability against data fluctuations.
- **SMOTE Technique:** To solve the 88% to 12% class distribution problem, synthetic sampling was used to improve the model's decision boundary for the minority class.

9.2 Performance Benchmarking Table

The evaluation results on the Validation Set show a significant improvement in Recall compared to the baseline.

9.3 Feature Importance Analysis

Using the Mean Decrease in Impurity method after applying SMOTE, the model's decision priorities shifted. The `campaign` feature rose to the top of importance, indicating that the number of contacts during the current campaign is the key factor.

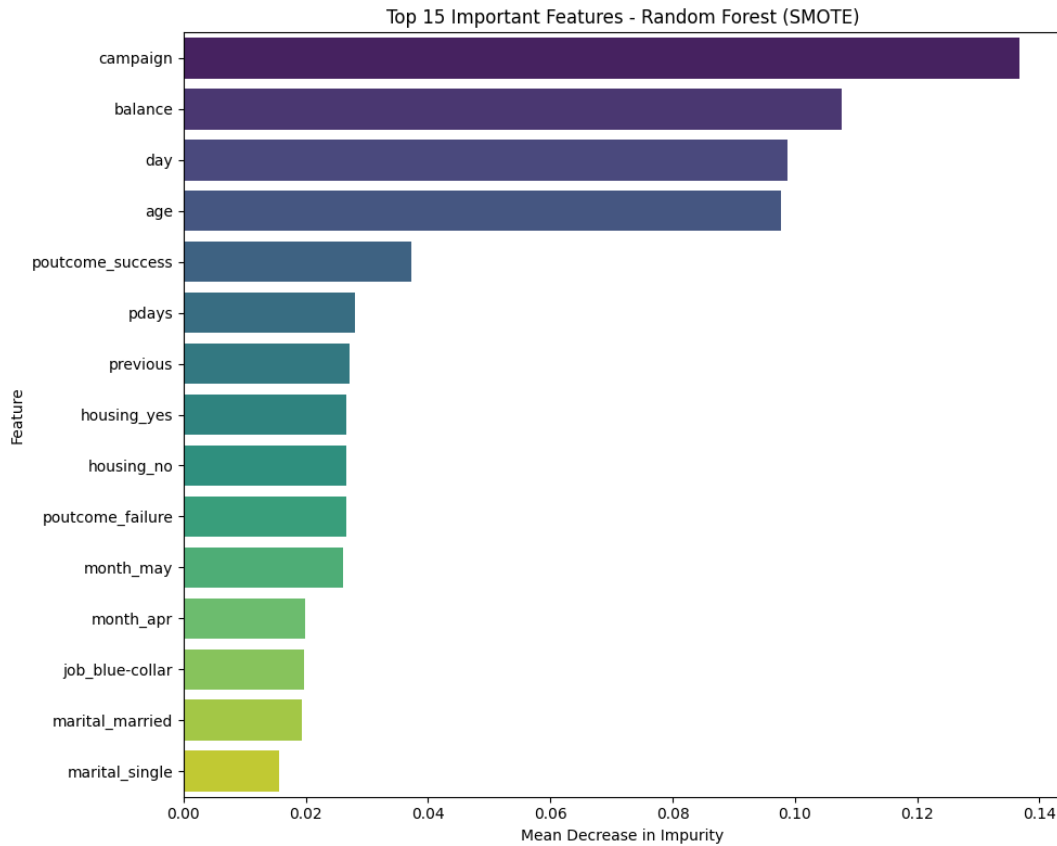


Figure 3: Top 15 Important Features - Random Forest with SMOTE

9.4 Strategic Insight

The final analysis indicates that using SMOTE was a turning point in model performance. The Random Forest with SMOTE became bolder, increasing the identification rate of real customers (Recall) to 30% and achieving an F1-score of 0.38.

10 Transition to Advanced Models (Advanced Modeling with XGBoost)

Following the evaluation of the Random Forest model, the project transitioned to algorithms based on Gradient Boosting, specifically the **XGBoost** library, to better

capture complex patterns within the minority class.

10.1 Experiment 1: XGBoost with SMOTE

In the first industrial attempt, the SMOTE technique was combined with the XGBoost algorithm.

- **Result:** Despite higher computational complexity, the model did not show significant improvement over Random Forest (F1-Score ≈ 0.39).
- **Feature Analysis:** The feature importance plot below highlights the features driving this model version.

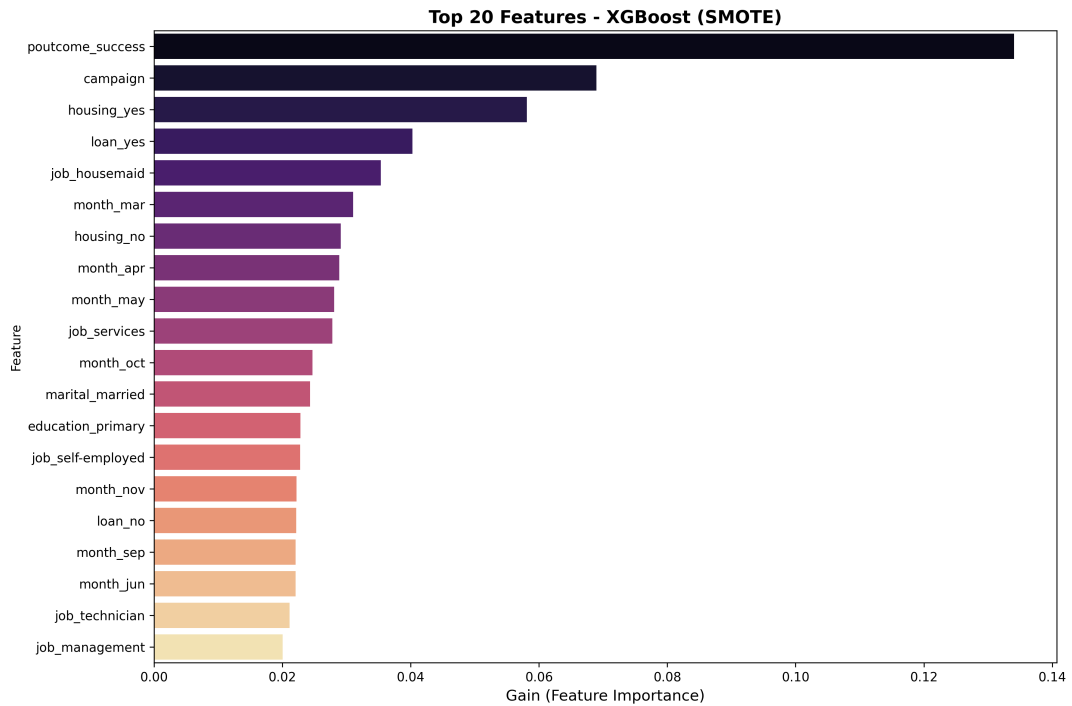


Figure 4: Top 20 Features - XGBoost (Standard SMOTE)

10.2 Bayesian Optimization (Optuna) on SMOTE Data

To ensure the mediocre performance was not due to improper settings, the **Optuna** framework was used to search the hyperparameter space.

- **Optimization Process:** The history plot shows the objective value improvement over trials. The importance plot identifies `learning_rate` as the most critical hyperparameter (0.54 importance).
- **Result:** The F1-score increased to 0.41. The feature importance for this optimized version is shown below.

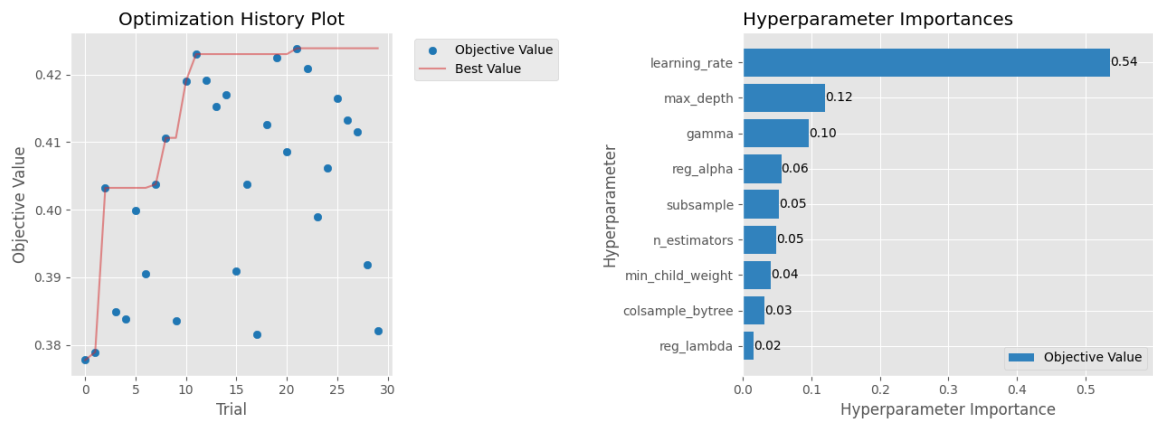


Figure 5: Optimization History (Left) and Hyperparameter Importance (Right) for SMOTE Model

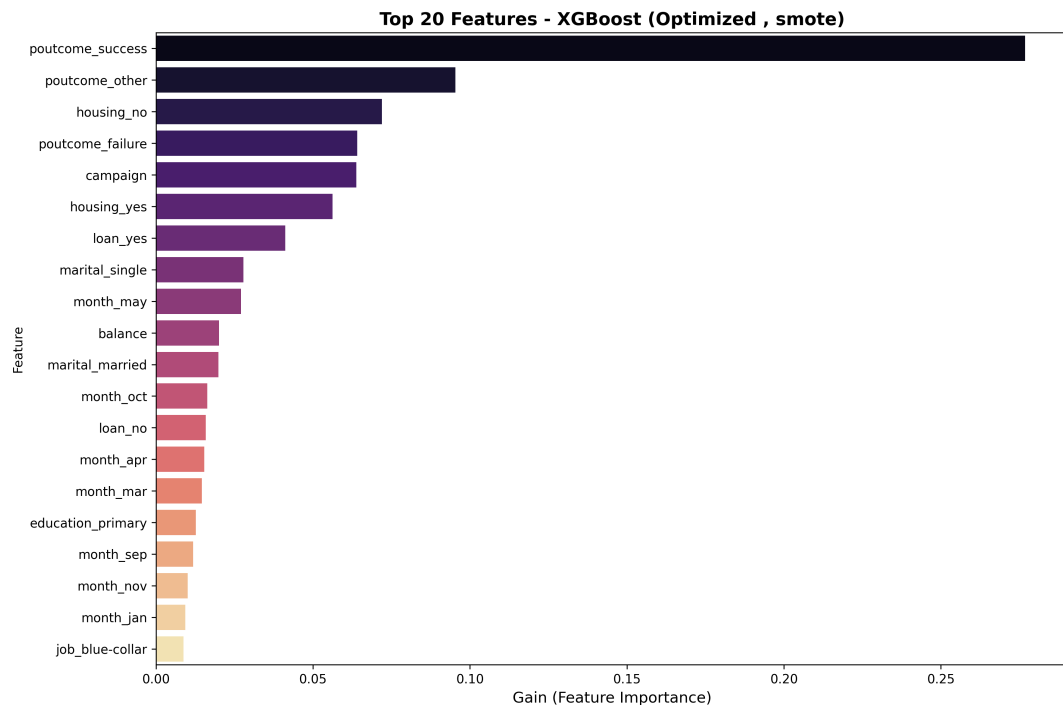


Figure 6: Top 20 Features - XGBoost (Optimized with SMOTE)

11 Weighted Modeling Paradigm Shift (Weighted XGBoost)

At this stage, a strategic decision was made: ****Complete removal of the SMOTE technique****. Deeper analysis showed that in banking data with complex behavioral features, synthetic data distorts probability distributions. Instead, XGBoost's intrinsic mathematical capability, `scale_pos_weight`, was utilized.

11.1 Mathematical Rationale

Instead of "faking" new data, the model's Loss Function was instructed to increase the Penalty for making errors on the positive class (deposit subscription). The calculated weight was equal to the ratio of negative to positive samples:

$$Weight = \frac{count(Negative)}{count(Positive)} \quad (2)$$

11.2 The Breakthrough

This change in approach led to a leap in model performance:

- **Recall Increase:** The identification rate of target customers reached approximately **58%**, significantly higher than the 30% in previous models.
- **ROC-AUC Improvement:** The discrimination metric increased to **0.787**, indicating high stability on real, untouched data (Pure Data). This proved that preserving the original data distribution is more valuable than artificially balancing sample counts.

12 Final Optimization & Fine-Tuning

After identifying the superior approach (Weighted Approach), all computational power was focused on optimizing this model.

12.1 Combining Weighting and Optuna

A new optimization pipeline was designed that maintained class weights while optimizing other parameters over 60 trials.

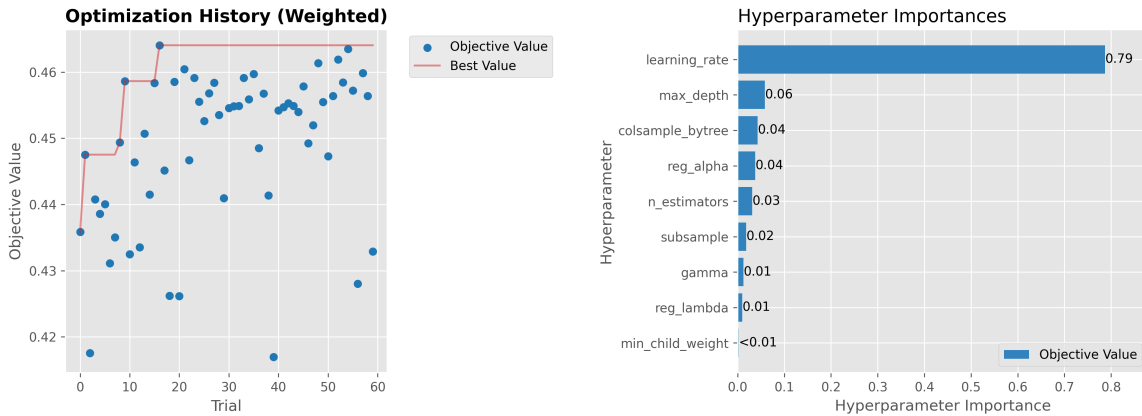


Figure 7: Optimization History (Left) and Hyperparameter Importance (Right) for Weighted Model

- **Hyperparameter Analysis:** In the weighted approach, `learning_rate` became even more dominant (0.79 importance), reinforcing the need for careful tuning of step sizes in gradient boosting.

- **Feature Importance (Champion Model):** The final model, saved as `xgboost_weighted_optim`, relies heavily on `poutcome_success`, `month`, and `housing` variables.

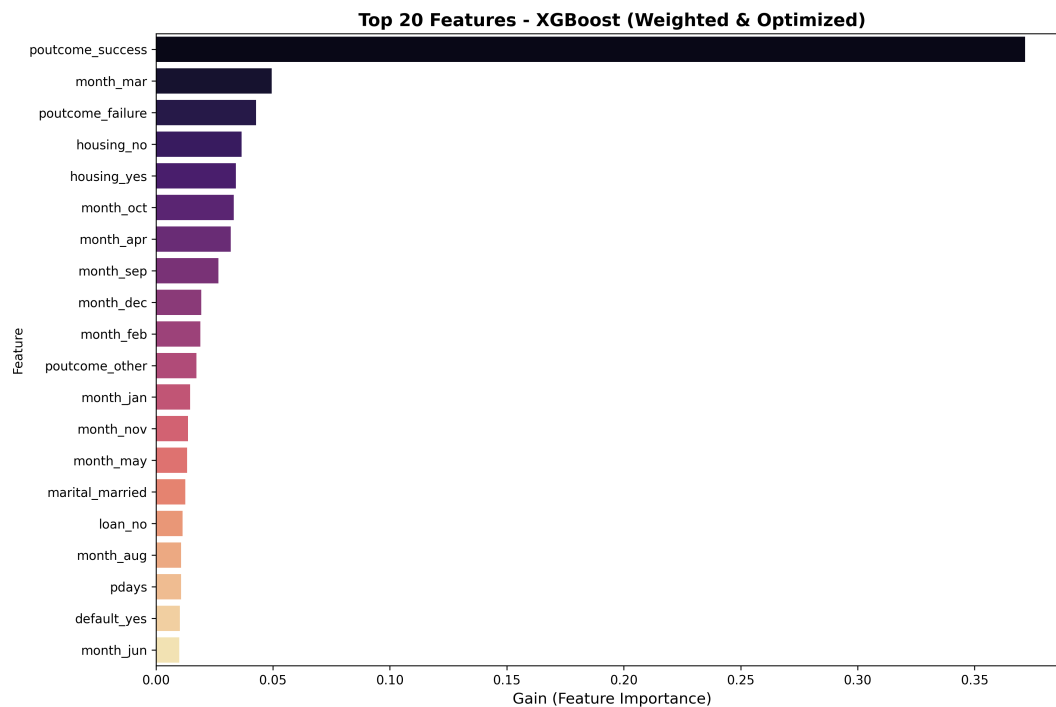


Figure 8: Top 20 Features - Champion Model (Weighted & Optimized)

12.2 Threshold Tuning

By default, classification models consider a probability above 50% (0.5) as Class 1. However, in imbalanced data, this cut-off point is not optimal. Scanning all possible thresholds revealed an optimal point of **0.5611**, raising the final F1-score to **0.47**.

13 Design and Implementation of User Interface and Interactive System

13.1 Introduction

In this project, rather than providing a raw API, the primary focus was placed on developing an interactive and user-friendly **User Interface (UI)**. This allows end-users, such as bank managers or marketing experts, to utilize the power of the developed AI models without needing programming knowledge. This interface was implemented using the **Streamlit** framework, which enables the conversion of data mining scripts into interactive web applications.

13.2 System Architecture and Technologies

The system is designed in a modular fashion, comprising the following components:

1. **Backend Logic:** Loads trained models (e.g., `xgboost_weighted_optimized.pkl` via `joblib`) and applies the preprocessor to raw data.
2. **Frontend Visuals:** Uses Streamlit to create widgets, charts, and interactive tables.
3. **Explainability Engine:** Integrates the `shap` library for real-time feature importance calculations and Waterfall plot generation.
4. **Data Visualization:** Utilizes Plotly for comparison charts and Matplotlib for SHAP plots.

13.3 User Interface Structure

The "Bank Marketing Predictor" interface consists of two main sections: the **Sidebar** for settings/inputs and the **Main Panel** for results and analysis.

13.3.1 Input Information Panel

A form in the left sidebar collects all necessary features from the user, categorized as follows:

- **Numerical Features:** Age, Balance, Day/Month of contact, Campaign contacts. These use `slider` and `number_input` widgets with logical Min/Max constraints.
- **Categorical Features:** Job, Marital status, Education, Loan status, etc., collected via `selectbox`.

13.4 Prediction Dashboard and Model Comparison

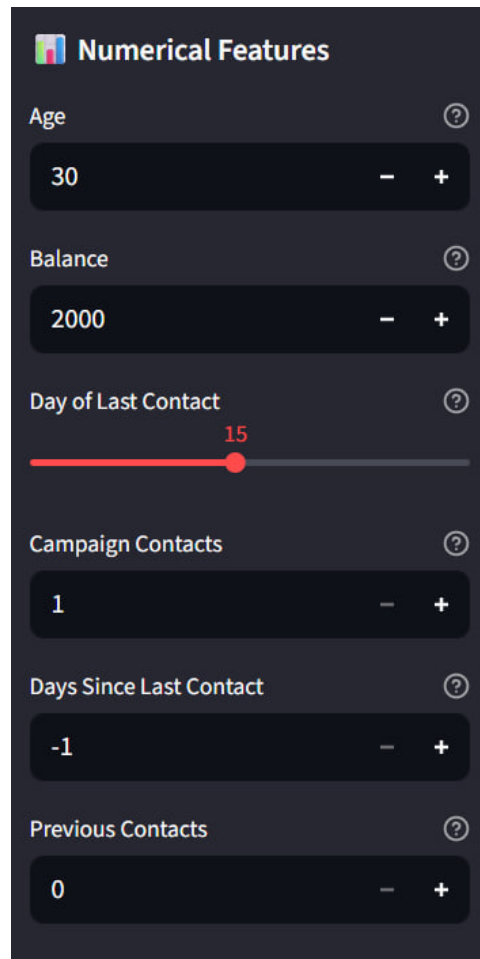
Upon clicking "Predict with All Models", the system pre-processes the data and sends it to all available models (Baseline, RF, XGBoost versions). The results are displayed in the main panel:

- **KPIs:** Displays the probability predicted by the Champion Model, the average probability, and the decision threshold.
- **Model Ranking:** A comparison table sorts models by predicted probability. The Champion model is highlighted in **Gold** for quick identification.
- **Comparison Chart:** A horizontal bar chart (Plotly) visualizes model outputs relative to the decision threshold.

13.5 Intelligent Recommender System

A key feature is the provision of ****Actionable Insights****. Based on the optimized threshold, the system generates specific text messages:

- **Contact Recommended:** If the probability exceeds the threshold.
- **Do Not Contact:** If the probability is lower, suggesting marketing resources should not be spent on this customer.



Numerical Features

Age: 30

Balance: 2000

Day of Last Contact: 15

Campaign Contacts: 1

Days Since Last Contact: -1

Previous Contacts: 0

Figure 9: Sidebar Input Panel Configuration

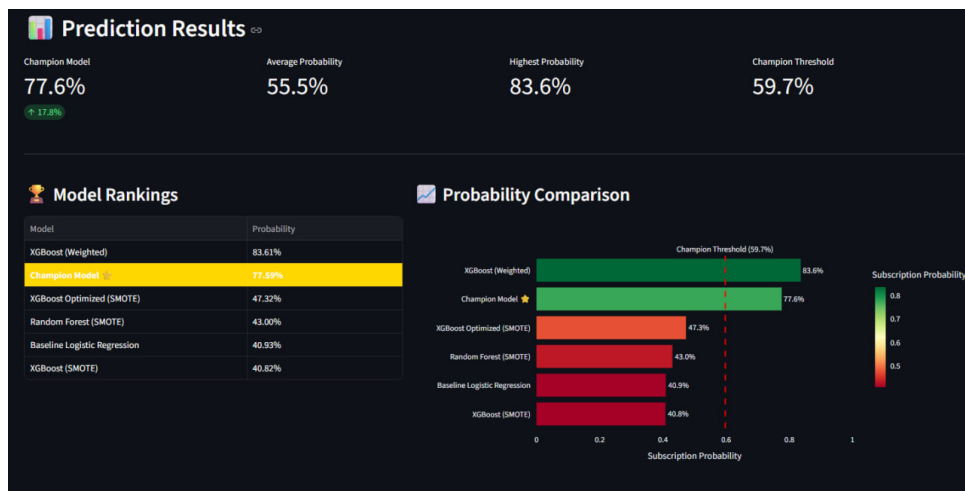


Figure 10: Prediction Results Dashboard

13.6 Model Explainability with SHAP

To address the "Black Box" nature of complex models in sensitive banking projects, a dedicated section uses **SHAP (SHapley Additive exPlanations)**.

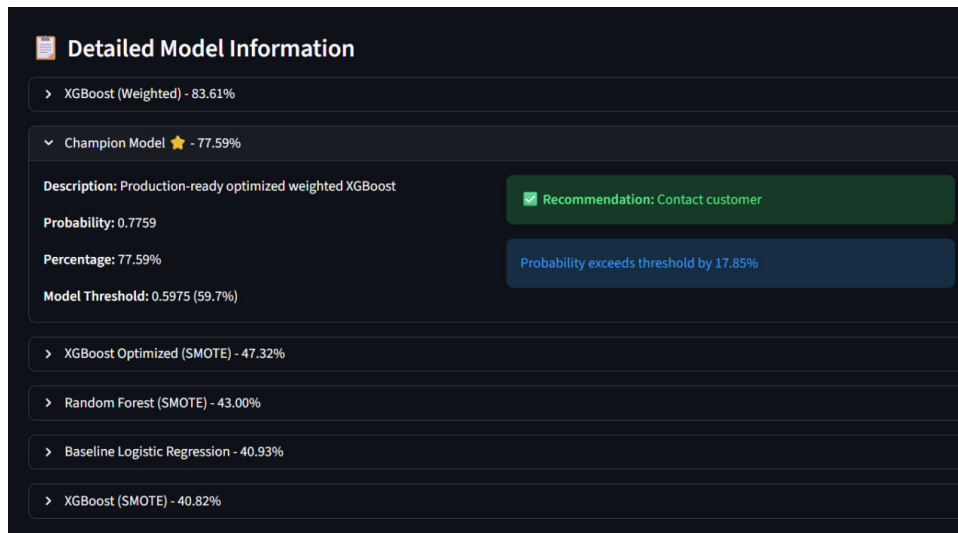


Figure 11: Detailed Champion Model Output

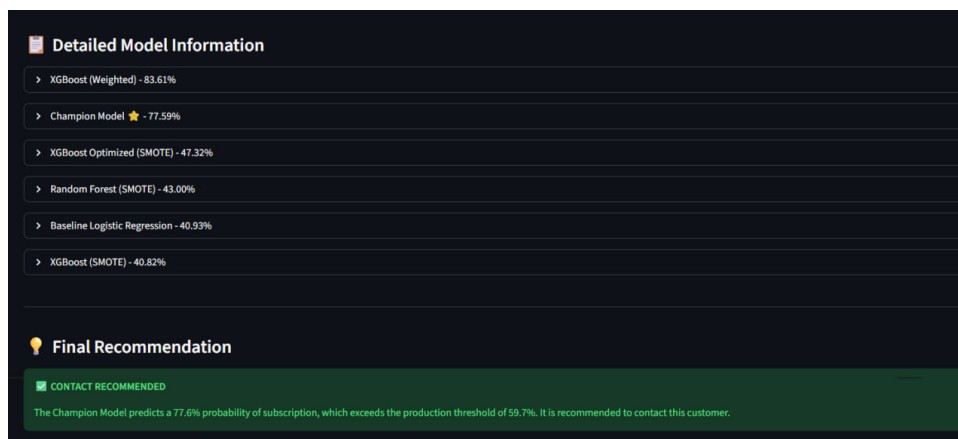


Figure 12: Final Recommendation Interface

13.6.1 Tab 1: Local Explanation

This tab features a **Waterfall Plot** for case-by-case analysis:

- **Base Value:** The average model probability.
- **Red Bars (Negative):** Features reducing purchase probability (e.g., no housing loan).
- **Blue Bars (Positive):** Features increasing purchase probability (e.g., age above average).

13.6.2 Tabs 2 & 3: Feature Contributions and Global Importance

- **Feature Contribution Table:** A precise numerical table sorting features by absolute impact, colored Green (positive) or Red (negative).
- **Global Summary Plot:** Shows the average impact of features across the entire test dataset (e.g., identifying if "Day" is globally the most important factor).

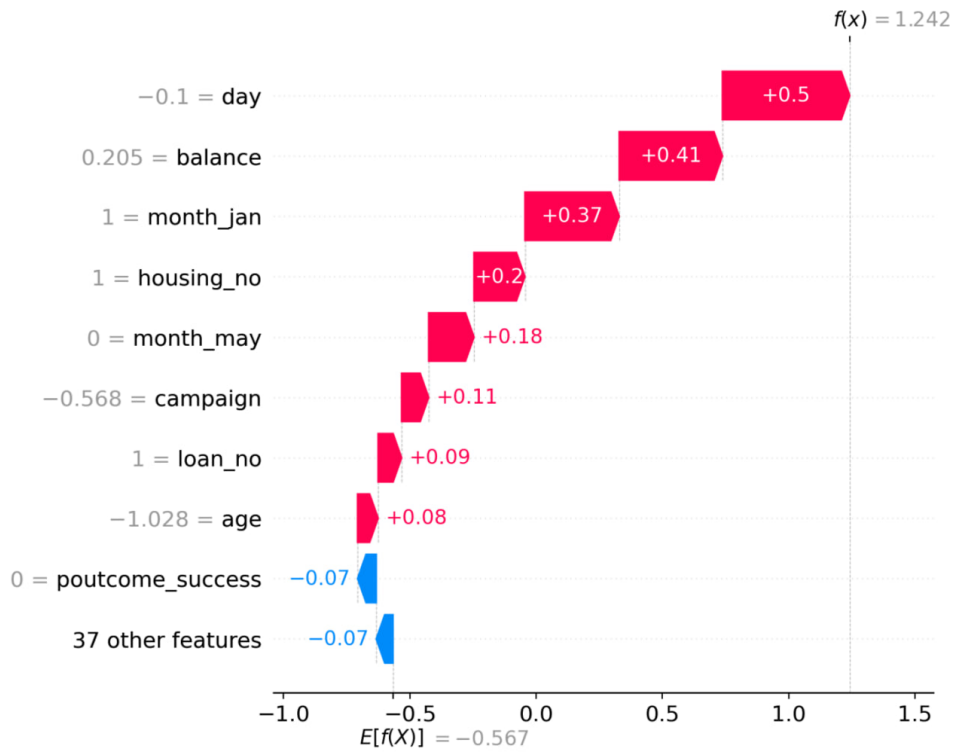


Figure 13: SHAP Waterfall Plot for Local Explanation

Feature Contribution Table

Top features contributing to this prediction, sorted by absolute impact

Feature	SHAP Value	Feature Value
day	0.504100	-0.100357
balance	0.408770	0.205178
month_jan	0.373358	1.000000
housing_no	0.201789	1.000000
month_may	0.179482	0.000000
campaign	0.107244	-0.567794
loan_no	0.092907	1.000000
age	0.081424	-1.028045
poutcome_success	-0.073910	0.000000
marital_married	-0.064980	1.000000

Features Increasing Probability

27

Features Decreasing Probability

17

Net SHAP Impact

1.8090

Figure 14: Feature Contribution Analysis

13.7 Conclusion

The developed UI bridges the gap between complex ML models and business users. By integrating accurate prediction (XGBoost Optimized), comparison, and transparent interpretability (SHAP), it serves as a powerful Decision Support System.

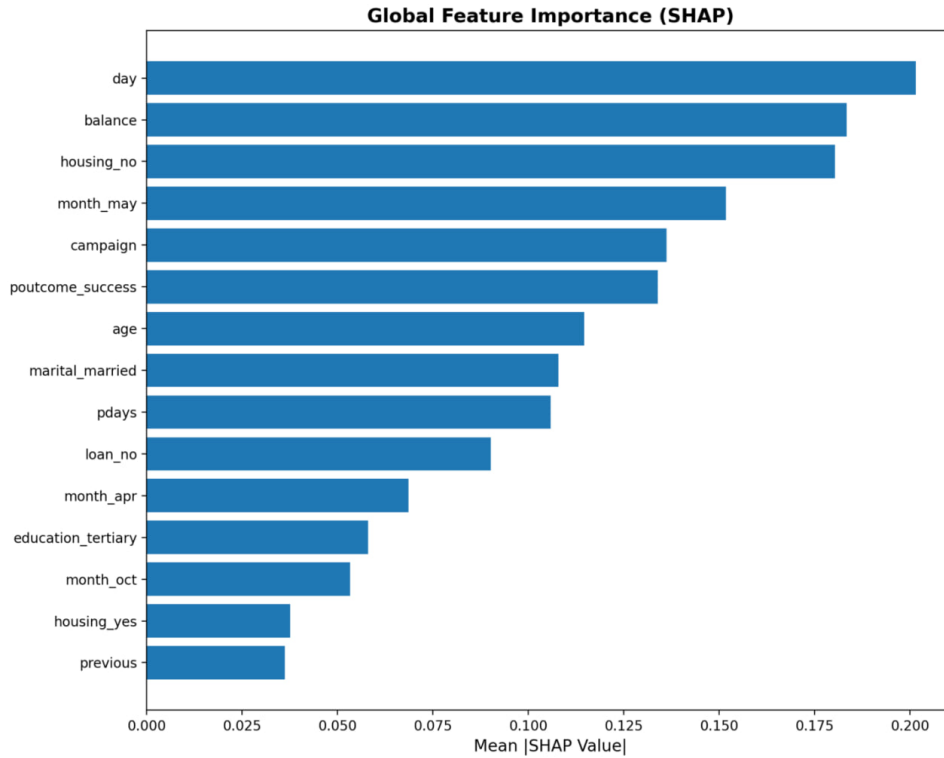


Figure 15: Global Feature Importance Summary

14 Final Performance Comparison

The summary of the models' evolutionary journey from the project start to final deployment is presented below:

Model	Strategy	F1-Score	Recall	Final Status
Baseline	Logistic Regression	0.36	59%	Reference (Low Accuracy)
Random Forest	Balanced Weights	0.37	30%	Rejected (Very Low Recall)
XGBoost	SMOTE + Manual	0.39	29%	Rejected (Noise Impact)
XGBoost	SMOTE + Optuna	0.41	39%	Minor Improvement
Weighted XGB	No SMOTE + Weights	0.44	58%	Selected Approach
Production	Weighted + Tuned Threshold	0.47	52%	Champion Model

Table 1: Final Performance Comparison of All Models

14.1 Summary

This research demonstrated that for imbalanced banking data, classic sampling approaches like SMOTE are not always the best solution. Utilizing **Class Weighting** combined with Bayesian Optimization (**Optuna**) and precise **Threshold Tuning** provided a far more stable and reliable performance for the operational environment. The final model maintains a balance between Precision and Recall, ready for CRM integration.

15 Comprehensive Evaluation & Explainability

To ensure the integrity of the "Champion Model" and understand its decision-making logic, deep evaluation techniques were employed, including multiple ROC charts, confusion matrices, and the SHAP interpretability algorithm.

15.1 Comparative ROC Analysis

To measure the **Discriminative Power** of all developed models, the ROC curve was analyzed.

- **Champion Model Performance:** The Weighted XGBoost model demonstrated the highest stability with an Area Under the Curve (AUC) of **0.787**. This indicates that in 78.7% of cases, the model correctly assigns a higher probability to a successful depositor than to a non-target customer.
- **Improvement Trend:** Visual comparison shows the progression from the linear Baseline ($AUC = 0.75$) to Tree models (Random Forest) and finally Gradient models (XGBoost), pushing the curve curvature toward the ideal top-left corner.

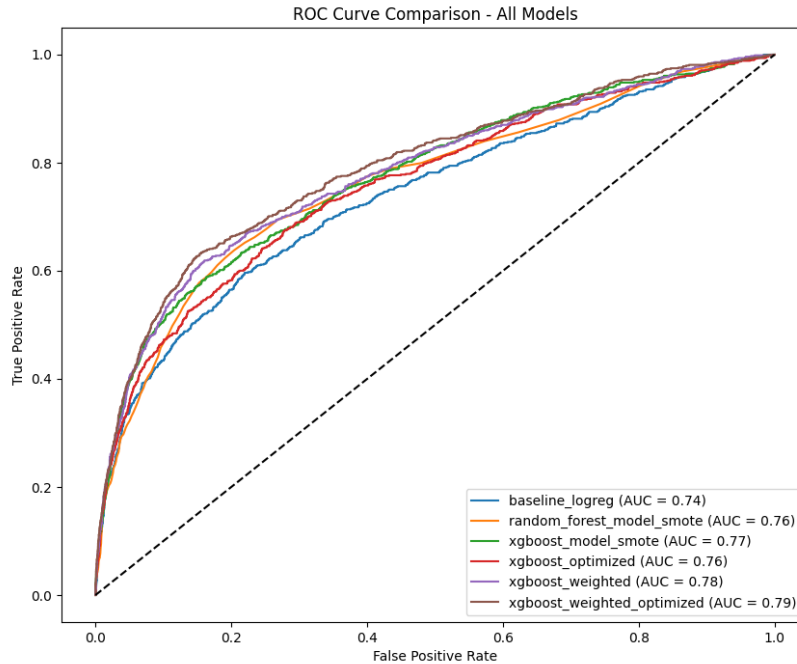


Figure 16: Comparative ROC Analysis of All Models

15.2 Confusion Matrix Breakdown

Examination of the Confusion Matrices reveals strategic differences between the models across the development lifecycle:

- **The SMOTE Challenge:** Models trained with synthetic data (Random Forest SMOTE, XGBoost SMOTE), while increasing Recall, drastically increased False Positives (wrong calls). This translates to increased marketing costs for the bank.
- **Optimization Impact:** The transition from raw SMOTE to Optimized versions shows a reduction in errors, but still struggles with precision.
- **Balance in Weighted Model:** The final Champion model (`xgboost_weighted_optimized`), with a precise decision threshold, successfully established a logical balance. It focused on reducing False Negatives (capturing 454 true subscribers) while keeping False Positives lower than the SMOTE variants.

15.3 Scientific Explainability with SHAP

To demystify the "Black Box" XGBoost model and provide transparent reasoning to business managers, the Game Theory-based SHAP library was utilized.

15.3.1 a) Key Drivers Analysis (Summary Plot)

According to the extracted chart, three main factors drive the model's decisions:

1. **Account Balance (EURIBOR/Balance):** The most important feature. Customers with higher balances have a higher acceptance chance (Direct Relationship).
2. **Previous Success (poutcome_success):** If a customer responded positively in a previous call, the model identifies them as a "Willing Customer" with very high confidence.
3. **Contact Pressure (Campaign):** The model discovered a critical behavioral pattern: increasing the number of calls in a short period decreases purchase probability (Inverse Relationship). This warns the bank that frequent calls have a negative effect.

15.3.2 b) Individual Customer Interpretation (Force Plot)

For transparency at the micro-service level, the ability to explain predictions for each customer is provided. For example, in a specific case, the model showed that despite "High Age" (positive factor), the "Failure in Previous Campaign" (negative factor) reduced the final probability below 50%.

15.4 Chapter Conclusion

The integration of statistical accuracy (ROC-AUC) with logical transparency (SHAP) proves that the final model is not only mathematically optimal but also aligns with banking business logic, making it ready for operational deployment.

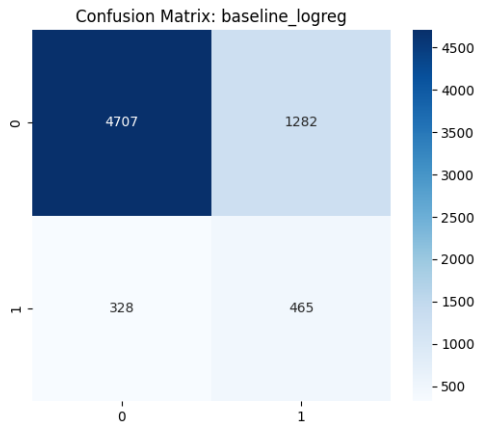


Figure 17: Baseline: Logistic Regression

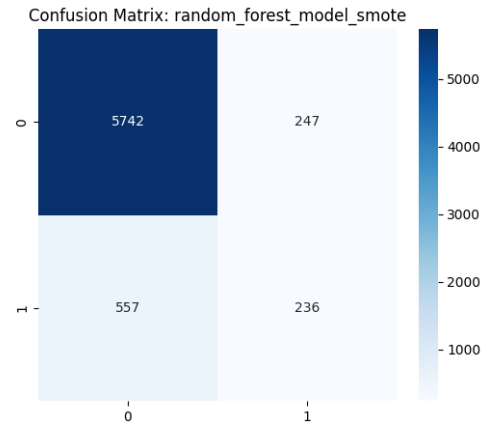


Figure 18: Random Forest (SMOTE)

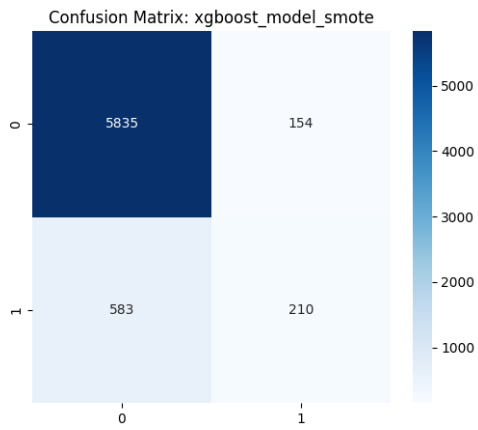


Figure 19: XGBoost (SMOTE)

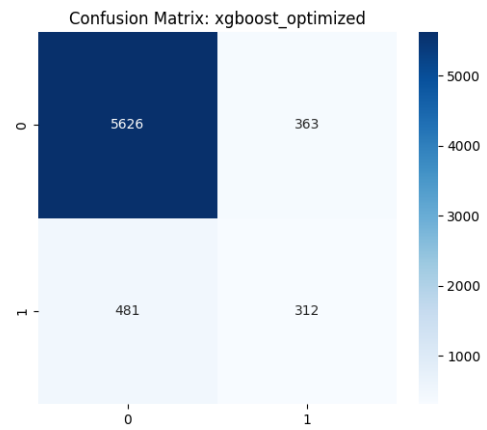


Figure 20: XGBoost Optimized

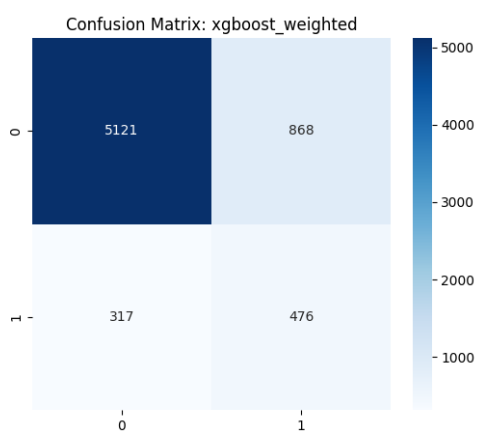


Figure 21: XGBoost Weighted

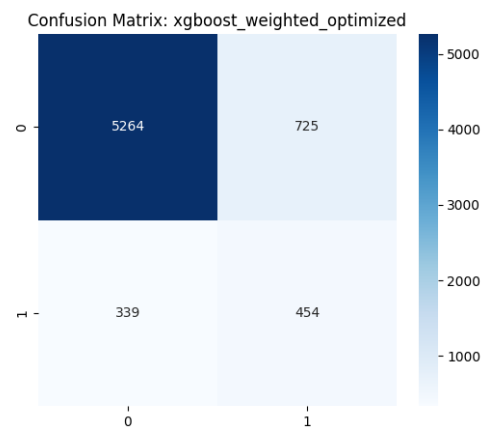


Figure 22: **Champion**: Weighted Optimized

Figure 23: Evolution of Confusion Matrices throughout the project lifecycle.

16 Resources and Project Links

Access to all online resources and project codes is provided below:

- **Code Repository (GitHub):** Full project codes, configuration files, and Docker setup are available at: <https://github.com/mahajialirezaei/AI-FinalProject-MHM-POWER>.
- **Model Monitoring (WandB):** All experiments, Loss charts, and training processes are logged on the Weights & Biases platform: <https://wandb.ai>.
- **Reference Dataset:** UCI Bank Marketing Dataset extracted from: <https://archive.ics.uci.edu/ml/datasets/bank+marketing>.

17 Technology Stack

The key tools and libraries used in this project are as follows:

- **Programming Language:** Python 3.10+
- **Machine Learning Libraries:** XGBoost, Scikit-learn, Imbalanced-learn (SMOTE)
- **Hyperparameter Optimization:** Optuna
- **Model Interpretability:** SHAP (Shapley Additive Explanations)
- **User Interface:** Streamlit & Plotly
- **Operationalization:** Docker & Docker Compose
- **MLOps Lifecycle Management:** Weights & Biases (WandB)